影像處理
--CH6_1

姓名:邱榮誠

學號:314512065

系級:電控碩 1

日期:2025/11/26

Q1.Please use a gradient computed in RGB color vector space to detect the edge for the image, 'lenna-RGB.tif' as Fig. 6.44(b) in pp.453. Please describe your method, procedures, final gradient image and print out the source code?

1）**source code**:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

img_path = Path(r"C:\Users\邱榮誠\Desktop\DIPHW_CH6-1\lenna-RGB.tif")
img = cv2.imread(str(img_path))
if img is None:
    from PIL import Image
    img = np.array(Image.open(img_path).convert("RGB"))
else:
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = img.astype(np.float64)
Gx_kernel = np.array([[-1, 0, 1],
                      [-2, 0, 2],
                      [-1, 0, 1]], dtype=np.float64)
Gy_kernel = np.array([[ 1,  2,  1],
                      [ 0,  0,  0],
                      [-1, -2, -1]], dtype=np.float64)
Gx = np.zeros_like(img)
Gy = np.zeros_like(img)
for c in range(3):   # R,G,B
    Gx[..., c] = cv2.filter2D(img[..., c], -1, Gx_kernel)
    Gy[..., c] = cv2.filter2D(img[..., c], -1, Gy_kernel)
grad = np.sqrt(np.sum(Gx**2 + Gy**2, axis=2))
grad = grad / grad.max() * 255
grad = grad.astype(np.uint8)
result_path = img_path.with_name("RGB_vector_gradient_b.png")
cv2.imwrite(str(result_path), grad)

plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
```

```
plt.imshow(img.astype(np.uint8))
plt.title("lenna-RGB")
plt.axis("off")
plt.subplot(1,2,2)
plt.imshow(grad, cmap='gray')
plt.title("final grdient image")
plt.axis("off")
plt.tight_layout()
plt.show()
```

2) **describe method:**

本次lab使用RGB Vector Gradient來偵測影像的邊緣，對應課本Figure 6.44(b)。一般的Sobel edge detection會先將影像轉成灰階，再計算梯度，本次方法則不同，直接對RGB三個通道各自做Sobel微分，再將三個通道的梯度透過向量方式合併，因此可同時偵測亮度變化與顏色變化所造成的邊緣。RGB向量梯度大小的計算公式如下：

$$|\nabla f(x,y)| = \sqrt{(G_x^R)^2 + (G_x^G)^2 + (G_x^B)^2 + (G_y^R)^2 + (G_y^G)^2 + (G_y^B)^2}$$

上式當中，(G_x^R), (G_x^G), (G_x^B)為對R、G、B三通道做SobelX方向微分。(G_y^R), (G_y^G), (G_y^B)為對R、G、B三通道做SobelY方向微分。

3) **describe procedures:**

- **讀取影像:**使用cv2.imread()讀取原始RGB影像。
- **建立Sobel微分濾波器:**定義SobelX與SobelY操作矩陣Gx_kernel與Gy_kernel。
- **對R、G、B三個通道分別進行Sobel微分:**使用cv2.filter2D()對每一通道進行濾波運算。得到每一通道的Gx與Gy。
- **以向量方式組合RGB的梯度:**將三個通道的X/Y梯度平方相加後取平方根，得到梯度大小$|\nabla f(x,y)|$。
- **輸出與顯示結果:**使用cv2.imwrite()儲存結果影像。使用matplotlib.pyplot.imshow()顯示最終梯度影像。

(4) final gradient image:

Q2.Repeat (1) steps in the image 'Visual resolution.gif'?

1)source code:

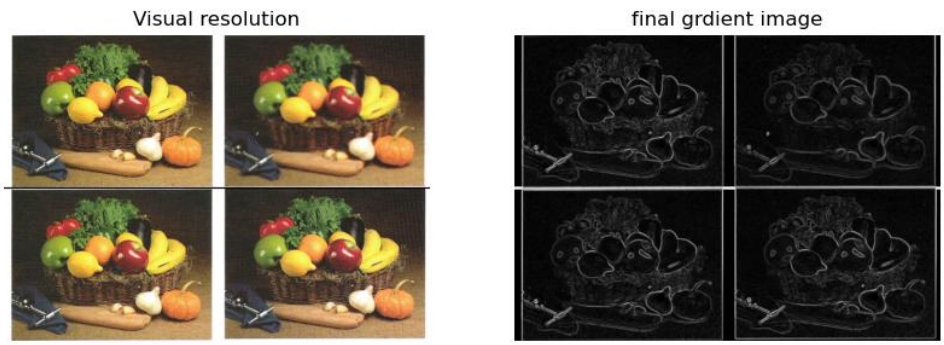
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path

img_path = Path(r"C:\Users\邱榮誠\Desktop\DIPHW_CH6-1\Visual
resolution.gif")
img = cv2.imread(str(img_path))
if img is None:
    img = np.array(Image.open(img_path).convert("RGB"))
else:
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = img.astype(np.float64)
Gx_kernel = np.array([[-1, 0, 1],
                      [-2, 0, 2],
                      [-1, 0, 1]], dtype=np.float64)
Gy_kernel = np.array([[ 1,  2,  1],
                      [ 0,  0,  0],
                      [-1, -2, -1]], dtype=np.float64)
```

```
Gx = np.zeros_like(img)
Gy = np.zeros_like(img)
for c in range(3):  # R,G,B
    Gx[..., c] = cv2.filter2D(img[..., c], -1, Gx_kernel)
    Gy[..., c] = cv2.filter2D(img[..., c], -1, Gy_kernel)
grad = np.sqrt(np.sum(Gx**2 + Gy**2, axis=2))
grad = (grad / grad.max() * 255).astype(np.uint8)
out_path = img_path.with_name("output_RGB_vector_gradient_b.png")
cv2.imwrite(str(out_path), grad)
print("Saved:", out_path)
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(img.astype(np.uint8))
plt.title("Visual resolution")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(grad, cmap='gray')
plt.title("final grdient image")
plt.axis("off")
plt.show()
```

2) describe method and procedures:

同第一題的(2)與(3)。

3) final gradient image:



Q3.Please comment and compare your two designs?

在Q1（Lenna）與Q2（Visual resolution）這兩題中，我所使用的方法完全相同，都是以RGB Vector Gradient的方式進行邊緣偵測。本方法直接對R、G、B 三個通道各自執行SobelX與Y微分，之後再以向量合併的方式計算梯度大小，因此能同時偵測亮度變化與顏色變化所產生的邊緣。因此，這是一種在RGB空間中進行梯度計算的方式，而非針對單一亮度資訊進行處理。

雖然兩題完全使用相同的演算法與程序，但結果呈現差異明顯。Lenna 影像具有較多細節、顏色差異以及明暗變化，因此在RGB向量梯度運算後，邊緣線條更清楚，人物輪廓、帽子與頭髮紋理都能明確被偵測出來。相比之下，Visual resolution影像內容包含大量顏色相近且邊界較柔和的區域，加上GIF壓縮造成的資訊損失，使得梯度變化不如Lenna鮮明，因此產生的邊緣圖較不清楚，邊緣線條也相對較少。可以得知顯示出影像內容特性會直接影響RGB梯度偵測的效果，即使採用相同的演算法，也會得到不同品質與強度的梯度結果。